

PCF Create, Build, Deploy

Cheat Sheet

The following are steps to create, build, and deploy PCF controls.

By Carl

3 min. read · [View original](#)

The following are steps to create, build, and deploy PCF controls. You will need to install the [Power Platform CLI](#) and have msbuild (from Visual Studio). Aliases are used where possible.

Usage: `pac pcf init [--namespace] [--name] [--template] [--framework] [--outputDirectory] [--run-npm-install]`

<code>--namespace</code>	The namespace for the component. (alias: -ns)
<code>--name</code>	The name for the component. (alias: -n)
<code>--template</code>	Choose a template for the component. (alias: -t) values: field, dataset
<code>--framework</code>	The rendering framework for control. The default value is 'none', which means HTML. (alias: -fw) values: none, react
<code>--outputDirectory</code>	Output directory (alias: -o)
<code>--run-npm-install</code>	Auto run 'npm

install' after the control is created. The default value is 'false'. (alias: -npm)

Initialize Project

How to initialize a project.

Field Templates

The following are **field** type templates.

Standard Field

Create Field (Standard template, non-React) without npm install (you would as an additional manual step):

```
pac pcf init -n <control name> -ns <namespace> -t field
```

<controlName> is used in the class in index.ts and in the constructor, display-name-key and description-key in the ControlManifest.input.xml.

<namespace> is used in the ControlManifest.input.xml.

Create Field (Standard template, non-React) with npm install:

```
pac pcf init -n <control name> -ns <namespace> -t field -npm
```

React (Virtual) Field

Create Field (React template) with npm install:

```
pac pcf init -n <control name> -ns <namespace> -t field -fw react -npm
```

Dataset Templates

The following are dataset templates.

Standard Dataset

Create Dataset (Standard template, non-React)
with npm install:

```
pac pcf init -n <control name> -ns <namespace> -t  
dataset -npm
```

React (Virtual) Dataset

Create Dataset (React) with npm install:

```
pac pcf init -n <control name> -ns <namespace> -t  
dataset -fw react -npm
```

Build and Package

Build with npm:

```
npm run build
```

Test harness and debug with npm:

```
npm start watch
```

Increment the version in the
ControlManifest.input.xml file if you are
deploying via a solution.

Create a subdirectory called **Solution**, then go
to that directory to run this command:

```
pac solution init --publisher-name developer --  
publisher-prefix dev
```

This creates a src subfolder in the solution
directory and a Solution.cdsproj file. The
src\Other directory has the files

Customizations.xml,
Relationships.xml, Solution.xml.

The publisher name is the name of the publisher that will be created (or used) by this command. The prefix provided will be used on the publisher and the control, e.g. if prefix is carlprefix_ then the control will be carlprefix_carlcontrolnamespace.carl_controlname.

Add reference to base (non-Solution) folder:

```
pac solution add-reference --path <path to base control folder>
```

This adds a “ProjectReference” back to ..\PCFBuildTest.pcfproj in the Solution.cdsproj file.

Run this also from Solution folder using the Visual Studio Developer command prompt:

```
msbuild /t:build /restore
```

This creates an **unmanaged** solution.zip file in the Solution\bin\Debug directory.

```
msbuild /p:configuration=release
```

This creates a **managed** solution (and smaller build) in the Solution\bin\release directory.

Deploy

If you have not created an authorization profile:

```
pac auth create -u <URL to Dynamics or Power Apps organization>
```

If you already have an authorization profile you can use below to view the list:

```
pac auth list
```

And below to select which org to deploy to:

```
pac auth select -i <index>
```

If pushing directly to an environment (without using a solution), run this from the base control directory (not Solution subfolder):

```
pac pcf push --publisher-prefix <publisher prefix>
```

If deploying a solution, be sure to increment the version in the ControlManifest.Input.xml file.

Folder Structure

Note the folder structure created above:

- Folder you created manually, e.g. C:\PCF\MyControl. This will contain config files such as package.json, package-lock.json, pcfconfig.json, tsconfig.json, .eslintrc.json. The .pcfproj file takes the name from this folder, e.g. MyControl.pcfproj
- Folder for **Control Name** specified above, e.g. if name is MyPCFControl, folder is called MyPCFControl. This can be a different name from the root folder above.
- node_modules, created as a result of **npm install**

— Solution, the directory you create to hold the solution. This gets populated when you run the solution commands, but that does not generate the solution.zip file.

— Solution\bin\Debug holds the unmanaged solution.zip file.

— Solution\bin\release holds the managed solution.zip file.

Errors

Error: Cannot find module
'ajv/dist/compile/codegen'

```
npm install --save-dev ajv
```

THANKS FOR READING. BEFORE YOU LEAVE, I
NEED YOUR HELP.

I AM SPENDING MORE TIME THESE DAYS
CREATING YOUTUBE VIDEOS TO HELP
PEOPLE LEARN THE MICROSOFT POWER
PLATFORM.

IF YOU WOULD LIKE TO SEE HOW I BUILD
APPS, OR FIND SOMETHING USEFUL
READING MY BLOG, I WOULD REALLY
APPRECIATE YOU SUBSCRIBING TO MY
YOUTUBE CHANNEL.

THANK YOU, AND LET'S KEEP LEARNING
TOGETHER.

CARL

<https://www.youtube.com/carldesouza>